

**Exercice 1** :**Problème de choix d'activité**

On se donne un entier naturel  $n$  et  $n$  couples  $(d_i, f_i) \in \mathbb{N}^2$  tels que pour tout  $i \in \llbracket 0; n-1 \rrbracket$ ,  $d_i < f_i$ .

$d_i$  et  $f_i$  représentent les instants de début et de fin de l'activité  $i$ . On cherche à planifier un maximum d'activités en évitant les incompatibilités, c'est à dire trouver un tuple  $(i_1, i_2, \dots, i_k)$  strictement croissant d'entiers entre 0 et  $n-1$  tel que :

- La contrainte  $\forall j \in \llbracket 1; k-1 \rrbracket, f_{i_j} \leq d_{i_{j+1}}$  est respectée (pas de chevauchement d'activités).
- On maximise  $k$  le nombre d'activités

On implémente le problème en C de la façon suivante :

- On généralise le problème à : *trouver un ensemble le plus grand possible d'activités commençant après une date  $m$* . Il suffit de prendre  $m = \min_{0 \leq i < n} d_i$  pour retrouver le problème d'origine.

- On définit **activite** un type correspondant à des couples d'entiers, et on représente les paramètres du problème comme un tableau de tels couples de taille  $n$ , trié<sup>1</sup> par ordre croissant des  $f_i$ , ainsi que l'entier  $m$ .

On représente une solution du problème par un tableau de  $n$  booléens indiquant pour chaque activité d'indice  $i$ , si cette activité a été sélectionnée.

- Pour  $m$  et  $((d_0, f_0), \dots, (d_{n-1}, f_{n-1}))$  donnés comme ci-dessus, le choix glouton consiste à prendre  $i_1$  le plus petit entier tel que  $d_{i_1} \geq m$  : en d'autres termes, parmi les activités qui commencent après la date  $m$ , on prend celle qui se finit le plus tôt.

Si aucun tel  $i_1$  n'existe, alors aucune activité n'est possible. et l'unique solution est  $()$ , qu'on représente par un tableau dont toutes les valeurs sont **false**.

- On prend  $m' = f_{i_1}$  comme nouvelle valeur de  $m$ , et on exclut tous les couples d'indice  $i \in \llbracket 0; i_1-1 \rrbracket$  car si  $f_i \leq f_{i_1}$ , alors en particulier  $d_i < f_{i_1}$ .

On fait un appel récursif avec  $m'$  et les portions des deux tableaux pour les indices à partir de  $i_1 + 1$ .

1. Implémenter l'algorithme ci-dessus avec une fonction récursive  
`void activites_glouton(int n, activite* horaires, int m, bool* choix, int i_min)` qui prend en argument un tableau de taille  $n$  représentant  $((d_0, f_0), \dots, (d_{n-1}, f_{n-1}))$ , la date de début minimale  $m$ , et un tableau de taille  $n$  dans lequel on indiquera les choix faits, et un indice minimal  $i_{\min}$  à partir duquel on considère les tableaux **horaires** et **choix**.
2. Écrire une fonction `bool* choix_activites (int n, activite* horaires)` qui résout le problème initial **toujours en supposant le tableau horaires trié**.
3. Montrer que l'algorithme utilisé renvoie bien une solution optimale du problème.
4. (a) Donner l'ordre de grandeur de la complexité de l'algorithme.  
 (b) Si on ne suppose pas le tableau des horaires trié, quel complexité serait nécessaire pour résoudre le problème ?

1. Par simplicité, on suppose le tableau déjà trié, pour ne pas avoir à écrire un tri pour le type qu'on vient de définir. Cependant, même si les fonctions d'ordre supérieur en C (utilisant des pointeurs de fonction) sont hors programme, vous pouvez faire avec `void qsort( void* ptr, size_t count, size_t size, int (*comp)(const void*, const void*))` l'équivalent de ce que `Array.sort` fait en OCaml avec une fonction de comparaison et un tableau.

**Exercice 2 :****Autre approche du problème avec les graphes**

On considère le même problème donné par  $((d_0, f_0), \dots, (d_{n-1}, f_{n-1}))$ , avec les  $f_i$  dans l'ordre croissant. On se donne  $G = (V, E)$  le graphe non orienté sur les sommets  $V = \llbracket 0; n-1 \rrbracket$  avec

$$\forall u, v \in V, \{u, v\} \in E \iff \begin{cases} u \neq v \\ ]d_u; f_u[ \cap ]d_v; f_v[ \neq \emptyset \end{cases}$$

On appelle un tel graphe un *graphe d'intervalles*. Le problème de choix d'activités revient à trouver un ensemble indépendant, c'est à dire un ensemble de sommets qui définit un sous-graphe induit sans arêtes, dans un graphe d'intervalles.

1. Montrer que si  $G = (V, E)$  est un graphe d'intervalles défini comme ci-dessus à partir d'intervalles trié par ordre croissant de leur borne supérieure, alors on a :

$$\forall u, v \in V \text{ tels que } u < v, \{u, v\} \in E \implies \forall w \in \llbracket u; v-1 \rrbracket, \{w, v\} \in E$$

2. Quelle représentation vous paraît la plus pertinente pour un tel graphe ? Pourquoi ?
3. En utilisant les fonctions des interfaces `graphes.h` et `listes.h`, écrire une fonction `bool* ensemble_indep_intervalles(graphe* g)` qui calcule un tel ensemble indépendant.
4. Montrer que, en faisant les bonnes suppositions sur l'implémentation des graphes, votre fonction a une complexité linéaire en le nombre de sommets du graphe dans le pire cas.

**Exercice 3 :****Attribution de salles pour des activités**

On se donne toujours un entier naturel  $n$ , et  $n$  couples  $(d_i, f_i) \in \mathbb{N}^2$  tels que pour tout  $i \in \llbracket 0; n-1 \rrbracket$ ,  $d_i < f_i$ .

On cherche maintenant à attribuer à chaque activité une salle (les salles sont indistinguables entre elles), de façon à minimiser le nombre total de salles utilisées, c'est à dire à trouver un  $n$ -uplet  $(s_0, \dots, s_{n-1}) \in \mathbb{N}^n$  tel que

- La contrainte  $\forall i \neq j \in \llbracket 0; n-1 \rrbracket, s_i = s_j \implies f_i \leq d_j \text{ ou } f_j \leq d_i$  est respectée (pas de chevauchement d'activités dans la même salle).
- On minimise le nombre de salles  $1 + \max_{0 \leq i \leq n-1} (s_i)$ .

**On suppose désormais les activités triées par date croissante de début.**

1. Comment reformuler le problème pour avoir une sous-structure ? Montrer que cette sous-structure est optimale.
2. Montrer que, dans le problème reformulé, on peut toujours modifier une solution optimale  $(s_0, \dots, s_{n-1})$  de façon à attribuer à l'activité d'indice 0 le plus petit entier  $s'_0$  qui respecte les contraintes, en gardant l'optimalité de la solution.
3. En déduire la correction l'algorithme glouton qui attribue à chaque étape à l'activité qui commence le plus tôt le plus petit entier qui est compatible selon les contraintes.